

Implementasi Sistem Pengamanan File Teks Menggunakan Algoritma AES-256 Berbasis Python

Aesya Ditya Safina¹

¹Program D3 Teknik Informatika, Politeknik Purbaya, Kabupaten Tegal,
aesyaditya6@gmail.com

Abstrak

Perkembangan teknologi informasi meningkatkan risiko kebocoran data sehingga diperlukan pengamanan yang efektif. Penelitian ini bertujuan mengimplementasikan algoritma Advanced Encryption Standard (AES-256) untuk mengamankan file teks menggunakan bahasa pemrograman Python. Implementasi dilakukan dengan mode Cipher Block Chaining (CBC) dan pembangkitan kunci menggunakan fungsi hash SHA-256. Pengujian dilakukan melalui proses enkripsi dan dekripsi pada file teks menggunakan password yang sama. Hasil menunjukkan file berhasil diubah menjadi ciphertext sehingga tidak dapat dibaca oleh pihak yang tidak berwenang. Proses dekripsi menggunakan password yang benar berhasil mengembalikan isi file ke bentuk semula tanpa perubahan data. Dengan demikian, algoritma AES-256 efektif diterapkan untuk pengamanan data.

Kata Kunci: Kriptografi, AES-256, Python, Enkripsi, Dekripsi

Informasi Artikel:

Dikirim :
Ditelaah :
Diterima :
Publikasi :

Juni – Juli 2026, Vol 1 (1) : hlm 1-8
©2026 Politeknik Purbaya.
All rights reserved.

1. PENDAHULUAN

Perkembangan teknologi informasi telah meningkatkan penggunaan data digital dalam berbagai bidang, seperti pendidikan, pemerintahan, maupun bisnis. Berbagai dokumen penting kini disimpan dalam bentuk file digital karena lebih mudah dikelola dan didistribusikan. Namun, kemudahan tersebut juga meningkatkan risiko kebocoran data akibat akses oleh pihak yang tidak berwenang. Oleh karena itu, diperlukan mekanisme pengamanan yang mampu menjaga kerahasiaan dan integritas data. Salah satu metode yang banyak digunakan untuk mengatasi permasalahan tersebut adalah kriptografi (Oktavani, Rizky, dan Gunawan, 2023; Baso dan Limbong, 2024).

Advanced Encryption Standard (AES) merupakan algoritma kriptografi simetris yang banyak diterapkan dalam sistem keamanan informasi karena memiliki tingkat keamanan yang tinggi dan proses komputasi yang efisien. AES memiliki tiga variasi panjang kunci, yaitu 128 bit, 192 bit, dan 256 bit. Di antara ketiganya, AES-256 memberikan tingkat keamanan yang lebih tinggi sehingga banyak digunakan untuk melindungi berbagai jenis data digital, termasuk dokumen dan file teks (Zulma, Seta, dan Yuniati, 2022; Indrayani, Ferdiansyah, dan Koprawi, 2025).

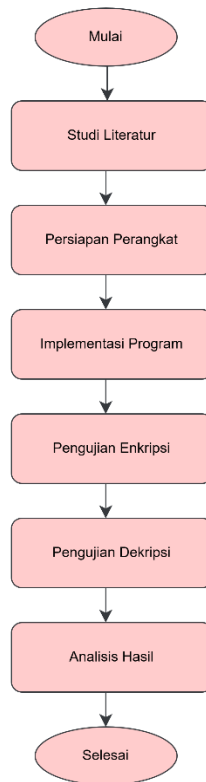
Beberapa penelitian terdahulu menunjukkan bahwa algoritma AES efektif digunakan dalam pengamanan data. Baso dan Limbong (2024) berhasil mengimplementasikan AES-256 untuk meningkatkan keamanan file, sedangkan Latip (2025) menerapkan AES pada pengamanan file teks dan memperoleh hasil bahwa algoritma tersebut mampu menjaga kerahasiaan data dengan baik. Penelitian lain oleh Bibiola, Kalsum, dan Alamsyah (2023) mengembangkan aplikasi pengamanan file berbasis web menggunakan AES, sementara Manullang, Allwine, dan Sembiring (2023) menerapkan AES dengan mode Cipher Block Chaining (CBC) untuk mengamankan file dokumen. Hasil penelitian tersebut menunjukkan bahwa AES mampu menghasilkan ciphertext yang tidak dapat dipahami tanpa proses dekripsi menggunakan kunci yang sesuai.

Berdasarkan penelitian-penelitian tersebut, implementasi AES lebih banyak diterapkan pada pengamanan dokumen digital atau aplikasi berbasis web. Penelitian ini berfokus pada implementasi algoritma AES-256 menggunakan bahasa pemrograman Python untuk mengamankan file teks melalui proses enkripsi dan dekripsi menggunakan mode Cipher Block Chaining (CBC). Pendekatan ini diharapkan dapat memberikan gambaran mengenai penerapan AES-256 sebagai metode pengamanan file teks yang sederhana namun efektif.

Penelitian ini bertujuan mengimplementasikan algoritma Advanced Encryption Standard (AES-256) dalam pengamanan file teks berbasis Python serta mengevaluasi hasil proses enkripsi dan dekripsi yang dilakukan. Hasil penelitian diharapkan dapat menjadi referensi dalam pengembangan aplikasi keamanan data berbasis kriptografi.

2. METODE

Penelitian ini menggunakan metode eksperimen dengan mengimplementasikan algoritma Advanced Encryption Standard (AES-256) untuk mengamankan file teks menggunakan bahasa pemrograman Python. Penelitian dilakukan melalui beberapa tahapan, yaitu studi literatur, persiapan perangkat, implementasi program, pengujian enkripsi dan dekripsi, serta analisis hasil. Alur penelitian ditunjukkan pada Gambar 1.



Gambar 1. Flowchart Penelitian

Studi literatur dilakukan dengan mempelajari berbagai referensi mengenai keamanan informasi, algoritma AES, mode Cipher Block Chaining (CBC), fungsi hash SHA-256, serta implementasi AES menggunakan bahasa pemrograman Python. Referensi diperoleh dari jurnal ilmiah, buku, dan dokumentasi PyCryptodome sebagai dasar dalam memahami mekanisme kerja algoritma sebelum dilakukan implementasi.

Tahap berikutnya adalah menyiapkan perangkat penelitian yang terdiri atas perangkat keras berupa laptop dengan sistem operasi Windows 11 serta perangkat lunak berupa Python 3.x, Visual Studio Code, PyCryptodome, Command Prompt, dan file teks (.txt) sebagai objek pengujian.

Implementasi dilakukan menggunakan bahasa pemrograman Python dengan memanfaatkan pustaka PyCryptodome. Algoritma AES-256 diterapkan menggunakan mode Cipher Block Chaining (CBC), sedangkan pembentukan kunci dilakukan menggunakan fungsi hash SHA-256 berdasarkan password yang dimasukkan oleh pengguna.

Pengujian dilakukan dengan mengenkripsi file teks menggunakan password tertentu, kemudian mendekripsi kembali file tersebut menggunakan password yang benar maupun password yang berbeda. Hasil dekripsi dibandingkan dengan file asli untuk mengetahui keberhasilan implementasi algoritma AES-256. Adapun skenario pengujian yang digunakan ditunjukkan pada Tabel 1.

Tabel 1. Skenario Pengujian

No	Pengujian	Hasil yang Diharapkan
1	Enkripsi file teks	File berubah menjadi ciphertext
2	Dekripsi password benar	File kembali seperti semula

3	Dekripsi password salah	File tidak dapat dipulihkan
---	-------------------------	-----------------------------

Selanjutnya dilakukan analisis terhadap hasil implementasi dengan membandingkan kondisi file sebelum enkripsi, setelah enkripsi, dan setelah dekripsi. Analisis dilakukan untuk mengetahui keberhasilan algoritma AES-256 dalam menjaga kerahasiaan data sekaligus memastikan bahwa isi file tetap utuh setelah proses dekripsi menggunakan password yang benar.

3. HASIL DAN PEMBAHASAN

Implementasi sistem dilakukan menggunakan bahasa pemrograman Python dengan memanfaatkan pustaka PyCryptodome sebagai library utama dalam proses kriptografi. Program dikembangkan menggunakan Visual Studio Code dan menerapkan algoritma Advanced Encryption Standard (AES-256) dengan mode operasi Cipher Block Chaining (CBC). Pembentukan kunci dilakukan menggunakan fungsi hash SHA-256 berdasarkan password yang dimasukkan oleh pengguna. Program memiliki dua fungsi utama, yaitu `encrypt_file()` untuk melakukan proses enkripsi dan `decrypt_file()` untuk melakukan proses dekripsi file teks.

```

1 from Crypto.Cipher import AES
2 from Crypto.Util.Padding import pad, unpad
3 import hashlib
4
5 # ===== ENKRIPSI =====
6 def encrypt_file(input_file, output_file, password):
7     with open(input_file, 'rb') as f:
8         data = f.read()
9
10    key = hashlib.sha256(password.encode()).digest()
11    cipher = AES.new(key, AES.MODE_CBC)
12
13    ciphertext = cipher.encrypt(pad(data, AES.block_size))
14
15    with open(output_file, 'wb') as f:
16        f.write(cipher.iv + ciphertext)
17
18    print("File berhasil dienkripsi!")
19
20 # ===== DEKRIPSI =====
21 def decrypt_file(input_file, output_file, password):
22     with open(input_file, 'rb') as f:
23         file_data = f.read()
24
25     key = hashlib.sha256(password.encode()).digest()
26
27     iv = file_data[:16]
28     ciphertext = file_data[16:]
29
30     cipher = AES.new(key, AES.MODE_CBC, iv=iv)
31     plaintext = unpad(cipher.decrypt(ciphertext), AES.block_size)
32
33     with open(output_file, 'wb') as f:
34         f.write(plaintext)
35
36     print("File berhasil didekripsi!")
37
38 # ===== MENU =====
39
40 choice = input("Pilih (1 = Enkripsi, 2 = Dekripsi): ")
41
42 if choice == "1":
43     inp = input("Masukkan nama file: ")
44     out = input("Nama file hasil (.enc): ")
45     pw = input("Password: ")
46     encrypt_file(inp, out, pw)
47
48 elif choice == "2":
49     inp = input("Masukkan file .enc: ")
50     out = input("Nama file hasil: ")
51     pw = input("Password: ")
52     decrypt_file(inp, out, pw)
53
54 else:
55     print("Pilihan tidak valid!")
56

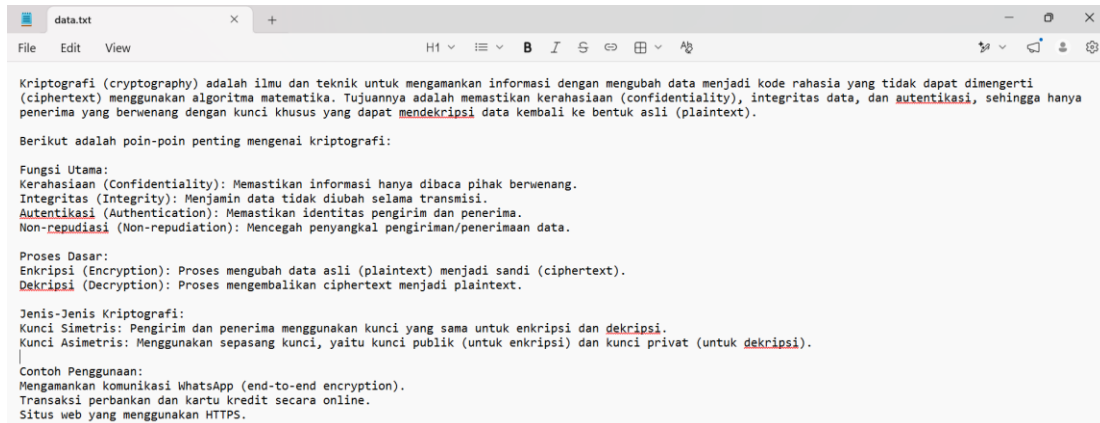
```

Gambar 2. Implementasi Program AES-256 Menggunakan Python

Berdasarkan Gambar 2, program berhasil mengimplementasikan proses enkripsi dan

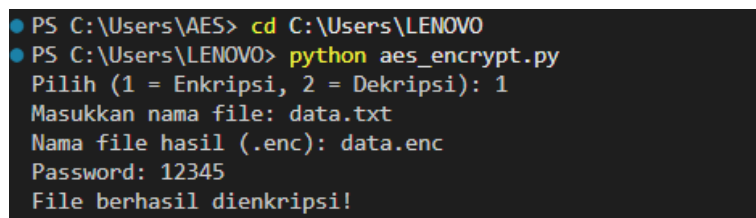
dekripsi file menggunakan algoritma AES-256. Program diawali dengan proses import library yang dibutuhkan, kemudian membangkitkan kunci menggunakan SHA-256 sebelum melakukan proses enkripsi atau dekripsi. Implementasi tersebut memungkinkan file teks diamankan sehingga hanya dapat dikembalikan menggunakan password yang sesuai.

Pengujian dilakukan menggunakan sebuah file teks sebagai objek penelitian. Sebelum proses enkripsi dilakukan, isi file masih berupa plaintext sehingga seluruh informasi dapat dibaca secara langsung menggunakan aplikasi editor teks.



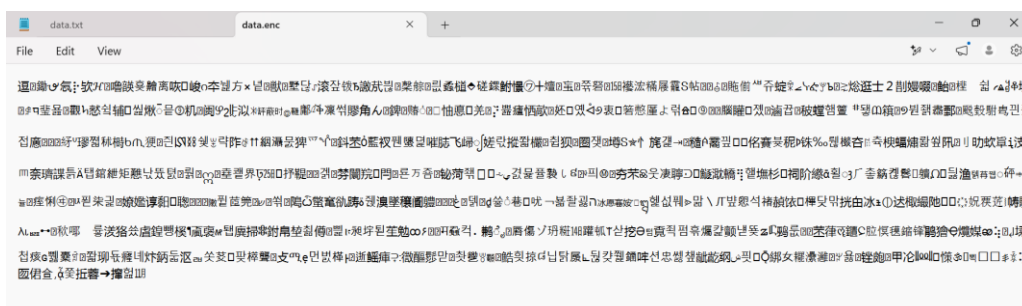
Gambar 3. File Teks Sebelum Enkripsi

Selanjutnya proses enkripsi dijalankan melalui Command Prompt dengan memasukkan nama file, nama file hasil enkripsi, serta password sebagai dasar pembentukan kunci.



Gambar 4. Proses Enkripsi File Menggunakan AES-256

Setelah proses enkripsi selesai, sistem menghasilkan file baru dengan ekstensi .enc yang berisi data dalam bentuk ciphertext. Perubahan isi file tersebut menunjukkan bahwa algoritma AES-256 berhasil mengubah data asli menjadi bentuk terenkripsi sehingga informasi tidak dapat dipahami tanpa melalui proses dekripsi.



Gambar 5. Hasil File Setelah Enkripsi

Tahap berikutnya adalah melakukan proses dekripsi menggunakan password yang sama dengan password pada saat proses enkripsi. Program membaca file hasil enkripsi, kemudian melakukan proses dekripsi sehingga isi file dapat dikembalikan ke bentuk semula apabila password yang dimasukkan sesuai.

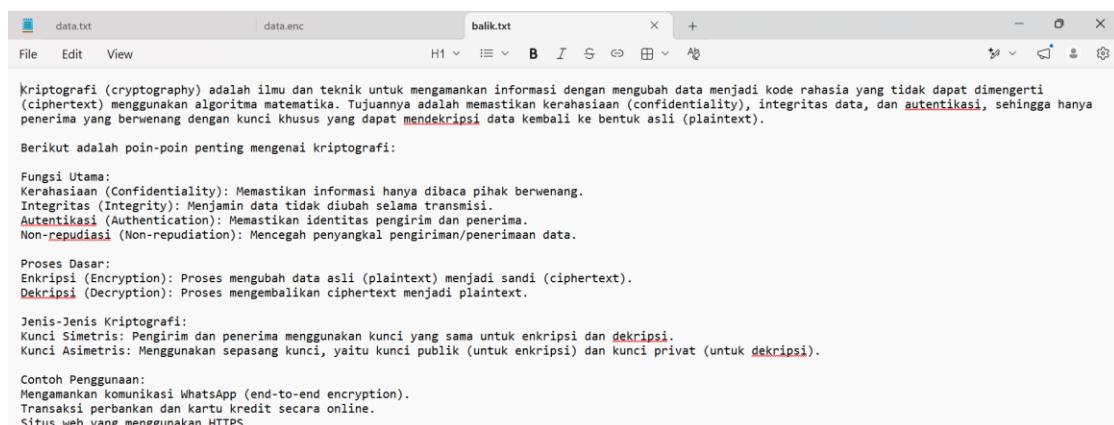
```

PS C:\Users\LENOVO> python aes_encrypt.py
Pilih (1 = Enkripsi, 2 = Dekripsi): 2
Masukkan file .enc: data.enc
Nama file hasil: balik.txt
Password: 12345
File berhasil didekripsi!
PS C:\Users\LENOVO>

```

Gambar 6. Proses Dekripsi File

Hasil dekripsi menunjukkan bahwa isi file kembali menjadi plaintext dan identik dengan file sebelum dilakukan proses enkripsi. Sebaliknya, apabila password yang dimasukkan tidak sesuai, maka sistem tidak dapat mengembalikan isi file ke bentuk semula sehingga file tetap tidak dapat dibaca. Hal ini menunjukkan bahwa keamanan file bergantung pada kerahasiaan password yang digunakan sebagai dasar pembentukan kunci.



Gambar 7. Hasil File Setelah Dekripsi

Berdasarkan seluruh proses pengujian, implementasi algoritma AES-256 berhasil mengamankan file teks melalui proses enkripsi dan dekripsi. Penggunaan mode Cipher Block Chaining (CBC) membuat setiap blok data saling berkaitan sehingga menghasilkan ciphertext yang lebih sulit dianalisis dibandingkan mode ECB. Selain itu, penggunaan fungsi hash SHA-256 menghasilkan panjang kunci sebesar 256 bit yang mendukung tingkat keamanan algoritma AES-256.

Hasil penelitian ini sejalan dengan penelitian Baso dan Limbong (2024) yang menyatakan bahwa implementasi AES-256 mampu meningkatkan keamanan file digital melalui proses enkripsi dan dekripsi. Penelitian ini juga mendukung hasil penelitian Manullang, Allwine, dan Sembiring (2023) yang menunjukkan bahwa penggunaan mode CBC mampu meningkatkan kerahasiaan data karena menghasilkan ciphertext yang tidak dapat dipahami tanpa proses dekripsi menggunakan kunci yang sesuai. Selain itu, penelitian Latip (2025) juga menunjukkan bahwa algoritma AES efektif diterapkan pada pengamanan file teks berbasis Python.

Tabel 2. Hasil Pengujian Implementasi AES-256

No	Parameter Pengujian	Hasil
1	Program berhasil dijalankan	Berhasil

2	Enkripsi file teks	Berhasil
3	Dekripsi dengan password benar	Berhasil
4	Dekripsi dengan password salah	Gagal membuka file
5	Kesesuaian file hasil dekripsi dengan file asli	Sesuai

Berdasarkan hasil pengujian tersebut dapat diketahui bahwa implementasi algoritma Advanced Encryption Standard (AES-256) berhasil diterapkan untuk mengamankan file teks menggunakan bahasa pemrograman Python. Program mampu menjaga kerahasiaan data melalui proses enkripsi dan mengembalikan isi file ke bentuk semula melalui proses dekripsi menggunakan password yang benar.

4. KESIMPULAN

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa algoritma Advanced Encryption Standard (AES-256) berhasil diterapkan untuk mengamankan file teks menggunakan bahasa pemrograman Python. Program mampu melakukan proses enkripsi dengan mengubah file plaintext menjadi ciphertext sehingga isi file tidak dapat dibaca secara langsung oleh pihak yang tidak memiliki password yang sesuai. Selain itu, proses dekripsi menggunakan password yang benar berhasil mengembalikan isi file ke bentuk semula tanpa mengalami perubahan isi data, sehingga kerahasiaan dan integritas informasi tetap terjaga.

Implementasi algoritma AES-256 menggunakan mode Cipher Block Chaining (CBC) serta pembentukan kunci melalui fungsi hash SHA-256 menunjukkan bahwa metode tersebut mampu memberikan tingkat keamanan yang baik dalam pengamanan file teks. Penelitian ini diharapkan dapat menjadi referensi dalam pengembangan aplikasi keamanan data berbasis kriptografi. Pada penelitian selanjutnya, program dapat dikembangkan dengan menambahkan antarmuka berbasis grafis (Graphical User Interface), dukungan terhadap berbagai jenis file, serta mekanisme validasi integritas file untuk meningkatkan keamanan dan kemudahan penggunaan.

DAFTAR PUSTAKA

- Baso, F., & Limbong, N. A. 2024. Implementasi teknik kriptografi dengan metode AES 256 untuk keamanan file. *INTEC Journal: Information Technology Education Journal* 3(3): 84–87.
- Bibiola, F., Kalsum, T. U., & Alamsyah, H. 2023. Penerapan algoritma Advance Encryption Standard (AES) untuk pengamanan file pada aplikasi berbasis web. *Jurnal Surya Energy* 8(1): 35–51. <https://doi.org/10.32502/jse.v8i1.6461>
- Hakim, A. R., & Margolang, K. F. 2025. Analisis keamanan data rekam medis digital menggunakan algoritma kriptografi AES. *Jurnal Teknologi Sistem Informasi dan Sistem Komputer TGD* 8(2): 108–114.
- Indrayani, R., Ferdiansyah, P., & Koprari, M. 2025. Analisis penggunaan kriptografi metode AES 256 bit pada pengamanan file dengan berbagai format. *Digital Transformation Technology (Digitech)* 4(2): 1245–1251. <https://doi.org/10.47709/digitech.v4i2.5457>
- Latip, P. N. 2025. Implementasi algoritma kriptografi AES dalam pengamanan file teks. *Jurnal Riset Sistem Informasi* 2(3): 1–4. <https://doi.org/10.69714/k6pr0s45>

- Manullang, S. F., Allwine, & Sembiring, J. 2023. Pengamanan data file dokumen menggunakan algoritma *Advanced Encryption Standard mode Cipher Block Chaining*. *ANTIVIRUS: Jurnal Ilmiah Teknik Informatika* 17(1): 53–67. <https://doi.org/10.35457/antivirus.v17i1.2811>
- Oktavani, S., Rizky, F., & Gunawan, I. 2023. Analisis keamanan data dengan menggunakan kriptografi modern algoritma *Advanced Encryption Standard (AES)*. *Jurnal Media Informatika (JUMIN)* 4(2): 97–101.
- Rukmana, K. T., & Ariyani, P. F. 2022. Penerapan algoritma AES-128 untuk pengamanan file pada SMK PGRI 31 Legok. *Prosiding Seminar Nasional Mahasiswa Fakultas Teknologi Informasi (SENAFTI)*: 327–336.
- Sutardi, & Wibowo, A. H. 2025. Implementasi AES 256-bit untuk enkripsi dan dekripsi teks serta dokumen Word. *ANIMATOR* 3(3): 17–25.
- Zulma, G. D. M., Seta, H. B., & Yuniati, T. 2022. Implementasi algoritma AES dan Bcrypt untuk pengamanan file dokumen. *Jurnal Informatik* 18(2): 163–176.